Library Management System (C++20)

A simplified library management system modelling **encapsulation, inheritance, abstraction, composition, aggregation and association**, built to modern C++20 practice: no raw pointers, ownership expressed through the type system, and SOLID applied throughout.

> **How to read this document.** Every reference below points at a **file and a symbol name** (`Book::SetTitle()`, `MyLibrary::UnregisterMember()`) rather than a line number, so the mapping stays correct as you edit the `.h` and `.cpp` files. Where a relationship matters more than a field's spelling, the **declared type** is given as well, since the type is what actually encodes composition versus aggregation.

Build and run

Requires a C++20 compiler and CMake 3.20+. Verified with MSVC 19.42 (Visual Studio 2022), CMake 3.29 and Ninja 1.12.

```
cmake -B build -G Ninja
cmake --build build
ctest --test-dir build --output-on-failure   # full unit-test suite
./build/library_demo                         # annotated walkthrough
```

On Windows the executables are `library_demo.exe` and `library_tests.exe`.

Warnings are errors (`/WX`, `-Werror`) on **all three** targets — library, demo and tests — via the `library_warnings` INTERFACE target in `CMakeLists.txt`. Disable with `-DLIBRARY_WARNINGS_AS_ERRORS=OFF`.

The test suite uses **GoogleTest**, fetched automatically by CMake's `FetchContent` on first configure (needs network access once; it's cached in `build/_deps` after that). Each `TEST()` is discovered at build time via `gtest_discover_tests()` and registered as its own CTest entry, so `CMakeLists.txt` never lists test names by hand.

Include style

`include/` is the include root, published by `target_include_directories(library_core PUBLIC include)`, so project headers are included by their path under it:

```
#include <library/MyLibrary.h>
```

The test target links `GTest::gtest_main` and includes `<gtest/gtest.h>`.

File map

File	Contains
`include/library/Book.h`, `src/Book.cpp`	`Book` — encapsulation
`include/library/Member.h`, `src/Member.cpp`	`Member`, `RegularMember`, `PremiumMember` — inheritance
`include/library/IBookRepository.h`	`IBookRepository` — the DIP seam
`include/library/InMemoryBookRepository.h`, `src/InMemoryBookRepository.cpp`	`InMemoryBookRepository` — wraps a standard container
`include/library/AbstractLibrary.h`	`AbstractLibrary` — abstraction
`include/library/MyLibrary.h`, `src/MyLibrary.cpp`	`MyLibrary` — composition, aggregation, association
`include/library/Results.h`, `src/Results.cpp`	result `enum class`es and their `to_string`
`app/main.cpp`	demo walkthrough
`tests/test_main.cpp`	unit tests (GoogleTest)

Requirement crib sheet

The four numbered steps of the brief, and exactly where each is satisfied.

Brief	Where
1. `Book` with private `title`, `author`, `isbn`	`Book::mTitle`, `Book::mAuthor`, `Book::mISBN` in `Book.h`
1. Getters and setters	`Book::Title()`, `Book::Author()`, `Book::ISBN()`, `Book::SetTitle()`, `Book::SetAuthor()`
2. `Member` with `name`, `member_id`, `list_of_borrowed_books`	`Member::Name()`, `Member::MemberId()`, `Member::ListOfBorrowedBooks()`
2. `borrow_book()` tracking borrowed books	`Member::BorrowBook()` in `src/Member.cpp`
2. `RegularMember` up to 3, `PremiumMember` up to 5	`RegularMember::kMaxBooks`, `PremiumMember::kMaxBooks`
3. Abstract library, a general concept	`AbstractLibrary` — pure virtual, cannot be instantiated
3. `add_book()`, `borrow_book()`, `return_book()`	the three pure virtuals declared on `AbstractLibrary`
4. Concrete `MyLibrary` implementing it	`class MyLibrary final : public AbstractLibrary`
4. Books and members in standard containers	`std::unordered_map` members of `MyLibrary` and `InMemoryBookRepository`

Brief	Where
4. `BookRepository` wrapping a standard container	`InMemoryBookRepository` delegates to `std::unordered_map`
4. References on **both** member and library side	`Member::ListOfBorrowedBooks()` and `MyLibrary`'s loan map, exposed via `MyLibrary::FindLoan()`
4. Register members, add, borrow, return	`MyLibrary::RegisterMember()`, `AddBook()`, `BorrowBook()`, `ReturnBook()`

The OOP concepts

Encapsulation — `Book`

All three attributes are private. Every mutating path runs through a setter that enforces the class invariant (no field may be empty or blank), so a `Book` cannot be driven into an invalid state — `Book::SetTitle()` and `Book::SetAuthor()` throw `std::invalid_argument` on blank input, as does the constructor. The shared guard is the `require_not_blank` helper in `src/Book.cpp`.

There is deliberately **no ISBN setter**. The ISBN is the book's identity and is used as the key of the repository's map; allowing mutation would silently desynchronise the container from its contents. Immutable identity is an encapsulation decision, not an omission.

`Member` encapsulates its own policy too: `Member::BorrowBook()` decides whether to accept a book and reports *why* it refused. The library asks; the member decides.

Inheritance — `Member` → `RegularMember` / `PremiumMember`

`Member` holds everything common. The subclasses vary exactly one thing: the value returned by `Member::MaxBooks()`.

Class	`MaxBooks()`
`RegularMember`	`RegularMember::kMaxBooks` (3)
`PremiumMember`	`PremiumMember::kMaxBooks` (5)

The limit is enforced once, in `Member::ReachedLimit()`, which the base class's `BorrowBook()` consults — so subclasses inherit enforcement for free and cannot forget it.

Abstraction — `AbstractLibrary`

"A Library is a general concept, not a tangible library." `AbstractLibrary` has three pure virtual methods, a virtual destructor, and **no data members** — an interface should not own state. All storage lives in `MyLibrary`.

Copy and move are `= deleted` on the polymorphic base to rule out slicing. `app/main.cpp` deliberately drives the library through an `AbstractLibrary&` (the `as_concept` reference) to show clients depend on the concept, not the concrete class.

Composition, aggregation, association — `MyLibrary`

All three are distinguished **by pointer type**, so the declaration of each member variable states its relationship:

Relationship	Owner → part	Declared type	Why
Composition	`MyLibrary` → repository	`std::unique_ptr<IBookRepository>`	Exclusive, cannot be shared, dies with the library
Composition	`InMemoryBookRepository` → books	`std::unordered_map<std::string, std::shared_ptr<Book>>`	Sole strong owner of every `Book`
Aggregation	`MyLibrary` → members	`std::unordered_map<int, std::weak_ptr<Member>>`	Library observes members it does not own
Association	`Member` → borrowed books	`std::vector<std::weak_ptr<Book>>`	Member references books it does not own
Association	`MyLibrary` → loans	`std::unordered_map<std::string, Loan>`	`MyLibrary::Loan` holds two `weak_ptr` — both ends, neither owned

`MyLibrary::Loan` is the bidirectional link: `Loan::mBook` and `Loan::mBorrower` are both non-owning, so the library observes the *relationship* between a book and a member without owning either.

The ownership model

> **Exactly one strong owner per object. Every other reference is `weak_ptr`.**

Apply that rule and the strong-reference graph is a forest — **acyclic by construction**. The classic `shared_ptr` cycle is not "broken" here; it never exists.

```
MyLibrary  --unique_ptr-->  IBookRepository    COMPOSITION
repository --shared_ptr-->  Book               COMPOSITION (sole owner)
client     --shared_ptr-->  Member             the client owns members
MyLibrary  --weak_ptr---->  Member             AGGREGATION
Member     --weak_ptr---->  Book               ASSOCIATION
MyLibrary  --weak_ptr x2->  Book + Member      ASSOCIATION (bidirectional)
```

The type system *is* the UML diagram: a `unique_ptr` member is the filled diamond, a `weak_ptr` is the plain line.

Why `weak_ptr` and not a raw pointer. A vanished object is *detected* (`expired()`) rather than dangling into undefined behaviour. Every use follows **promote → use → drop**: `lock()` once into a named local, use it, let it die. See `MyLibrary::LookupMember()` and `Member::HasBorrowed()`. The result of `lock()` is never stored, and never dereferenced unchecked.

Scope of "no raw pointers". No raw *owning* or *stored* pointer appears anywhere. `this` and references used as function parameters are unavoidable, and `const T&` is idiomatic non-owning access rather than a pointer.

Invariants that keep the two sides consistent

The association is stored twice (member side and library side), so it could drift. Three mechanisms prevent that:

1. **Validate before mutating.** `MyLibrary::BorrowBook()` performs *every* check — member registered, book exists, not already on loan, member accepts — before touching either side. A refusal cannot leave a half-written loan.
2. **Release per ISBN on departure.** `MyLibrary::UnregisterMember()` hands each book back through `Member::ReturnBook()` one ISBN at a time rather than clearing the member's whole list. This matters because the *client* owns the member: nothing stops it being registered with a second library, and those loans must survive.
3. **Self-healing sweep.** `MyLibrary::PruneDepartedMembers()` releases loans orphaned by a member destroyed without unregistering.

Two further guards worth noting:

- `MyLibrary::RemoveBook()` refuses to remove a book that is on loan (`RemoveBookResult::BookIsBorrowed`). That guard is what makes "the repository is sole owner" safe in practice.
- The loan map is keyed by ISBN, so the **container itself enforces** the business rule "at most one active loan per copy", and `MyLibrary::IsBorrowed()` answers availability in O(1).

SOLID

	Where
S — Single responsibility	`Book` is pure data + invariants and knows nothing about loans; `InMemoryBookRepository` does storage only; `Member` owns identity and its own borrowing policy; `MyLibrary` orchestrates. Loan bookkeeping lives in `MyLibrary`, never smeared into `Book`.
O — Open/closed	`Member::MaxBooks()` is the single extension point: a new `StudentMember` needs **zero** changes to `MyLibrary`. New storage backends plug in behind `IBookRepository`. The `switch` on `MemberBorrowResult` in `MyLibrary::BorrowBook()` is exhaustive, so adding a refusal reason is a compile error rather than a silent mislabel.
L — Liskov substitution	`RegularMember` and `PremiumMember` differ **only in the value** `MaxBooks()` returns. Neither strengthens a precondition, weakens a postcondition, nor throws where the base does not. The limit is a parameter of behaviour, not a change of contract, so any `Member&` is substitutable.
I — Interface segregation	`AbstractLibrary` carries exactly the three operations of the brief. Reporting and maintenance (`MyLibrary::RemoveBook()`, `FindLoan()`, `IsBorrowed()`, `PruneDepartedMembers()`, `GetRepository()`) live on the concrete class, so a borrow-only client never depends on them.

	Where
D — Dependency inversion	`MyLibrary` depends on `IBookRepository`, never on the concrete store, injected through `explicit MyLibrary(std::unique_ptr<IBookRepository>)`. `CountingRepository` in `tests/test_main.cpp` proves it by substituting an instrumented store.

Composition *and* DIP are not in conflict

A common objection: doesn't injecting the repository make it *aggregation*?

No. UML composition is about **exclusive lifetime ownership**, not about who calls `new`. Because the repository is held in a `std::unique_ptr`, it cannot be shared, cannot outlive the library, and cannot be reached by anyone else — the three defining properties of composition. Had it been a `shared_ptr`, *that* would be aggregation.

So both constructors express composition:

- `MyLibrary::MyLibrary()` — the whole creates its own part (textbook form).
- `MyLibrary::MyLibrary(std::unique_ptr<IBookRepository>)` — ownership is *transferred in*, satisfying DIP and making the class testable.

Error handling strategy

Three kinds of failure, three mechanisms, all declared in `Results.h`:

Mechanism	Used for	Examples
`enum class` result	Expected domain outcomes	`BorrowResult`, `ReturnResult`, `AddBookResult`, `RemoveBookResult`, `MemberBorrowResult`
`std::optional<T>`	Lookups that may find nothing	`MyLibrary::FindLoan()`
Exceptions	Broken invariants / programmer error	blank field in `Book`'s constructor; null argument to `MyLibrary::AddBook()`, `RegisterMember()`, or the injecting constructor

Why not `std::expected`? It is **C++23**, not C++20. The `enum class` choice is deliberate rather than a fallback: hitting a borrow limit is an ordinary path the demo exercises on purpose, and using exceptions for routine control flow would be an anti-pattern. Every result type has a `to_string` overload in `Results.h`, found by ADL so generic code can print outcomes.

Results are `[[nodiscard]]` — repeated on the `MyLibrary` overrides because the attribute is not inherited when calling through the derived type.

Test coverage

Every test lives in `tests/test_main.cpp` as a `TEST()` case, auto-discovered by `gtest_discover_tests()` -- no manual registration needed. Run them with `ctest --test-dir build --output-on-failure`, or run `library_tests` directly for GoogleTest's own output.

Test	Proves
`BookTest.Encapsulation`	Getters/setters work; invariants reject blank fields
`MemberTest.ReportsItsOwnRefusalReason`	`AlreadyHeld` / `LimitReached` / `BookUnavailable` are distinguished
`MemberTest.LimitsArePolymorphic`	Same base reference, different limits (LSP)
`MemberTest.RegularMemberCappedAtThree`	4th borrow refused; **no half-written loan left behind**
`MemberTest.PremiumMemberCappedAtFive`	6th borrow refused
`LibraryOperationsTest.AddBookRejectsDuplicateIsbn`	ISBN uniqueness enforced by the container
`LibraryOperationsTest.BorrowFailureModes`	Unknown ISBN, unregistered member, double borrow
`LibraryOperationsTest.ReturnBookFlow`	Wrong member, never-borrowed, return-twice
`LibraryOperationsTest.ReturnedBookFreesASlot`	Returning restores capacity
`AssociationTest.IsConsistentOnBothSides`	Member side and the loan map agree
`AggregationTest.LibraryDoesNotOwnItsMembers`	**Aggregation**: member destroyed in an inner scope, departure detected, book survives
`AggregationTest.RegisterAndUnregister`	Duplicate IDs, null rejection, loan release
`AggregationTest.UnregisteringLeavesNoStaleState`	Regression: departure leaves no stale claim on either side

Test	Proves
`AggregationTest.UnregisteringFromOneLibraryLeavesTheOtherIntact`	Regression: leaving one library preserves the other's loans
`CompositionTest.RepositoryIsSoleOwnerOfBooks`	**Composition**: book outlives the caller's handle, dies with the repository
`CompositionTest.RemoveBookRefusesWhileOnLoan`	Removal guard
`DipTest.RepositoryIsInjectable`	**DIP** via `CountingRepository`
`DipTest.UsableThroughTheAbstraction`	Full flow through `AbstractLibrary&`

The one to read first is **`AggregationTest.LibraryDoesNotOwnItsMembers`**: a member borrows a book, goes out of scope, and the library's `weak_ptr` expires. The departure is *detected* rather than dangling, the orphaned loan is released, and the book survives because it was never the member's to own. That single test is the proof the whole ownership model is right.

The demo

`./build/library_demo` prints a labelled walkthrough. Each section is designed so its output actually demonstrates its heading:

Section	Demonstrates
Stocking the shelves	`add_book`, and a duplicate ISBN rejected
Members arrive (aggregation)	Client owns members, library only observes
Regular member is capped at 3	Limit refused on an *available* book
Premium member is capped at 5	Identical code path, higher limit (OCP/LSP)
Expected failure modes	Each `BorrowResult` / `ReturnResult` reason
Returning frees a slot	Capacity restored, then reused
Association from the library side	Read back from the library via `IBookRepository::GetAll()`
A member leaves without returning	`weak_ptr` expiry detected; pruning drops the loan count
Books outlive their borrowers	Removal refused while on loan, allowed once free

---

Design decisions worth defending

Why is `Book` unaware of who borrowed it? SRP, and cycle avoidance. Loan state lives in `MyLibrary`. A `Book → Member` back-reference is the usual way a `shared_ptr` cycle gets introduced; keeping `Book` a pure data

abstraction means it has exactly one reason to change.

Why is the borrowed-books list a `vector` rather than a `set`? At most five entries, so a linear scan beats hashing, and insertion order is the natural display order. `Member::HasBorrowed()` guards against duplicates.

Why can `ListOfBorrowedBooks()` contain expired entries? A member is owned by the client, so it can outlive the library it borrowed from — and when a library dies it takes its repository, and therefore its books, with it. The entries are harmless: every operation locks before use and `Member::BorrowedCount()` ignores expired entries, so no borrow slot is ever consumed by a book that no longer exists. `BorrowedCount()` is the authoritative count, not `.size()`.

Why do `Member` and the interfaces delete copy and move? They are polymorphic bases, always handled through smart pointers, so deleting copy and move costs nothing and rules out slicing.

Why does `Member::BorrowBook()` return an enum instead of `bool`? So the member stays the single authority on its own policy. It evaluates the rule once and reports the reason; `MyLibrary` forwards it rather than re-deriving it, which means a future member type with different rules cannot make the reported outcome go stale.